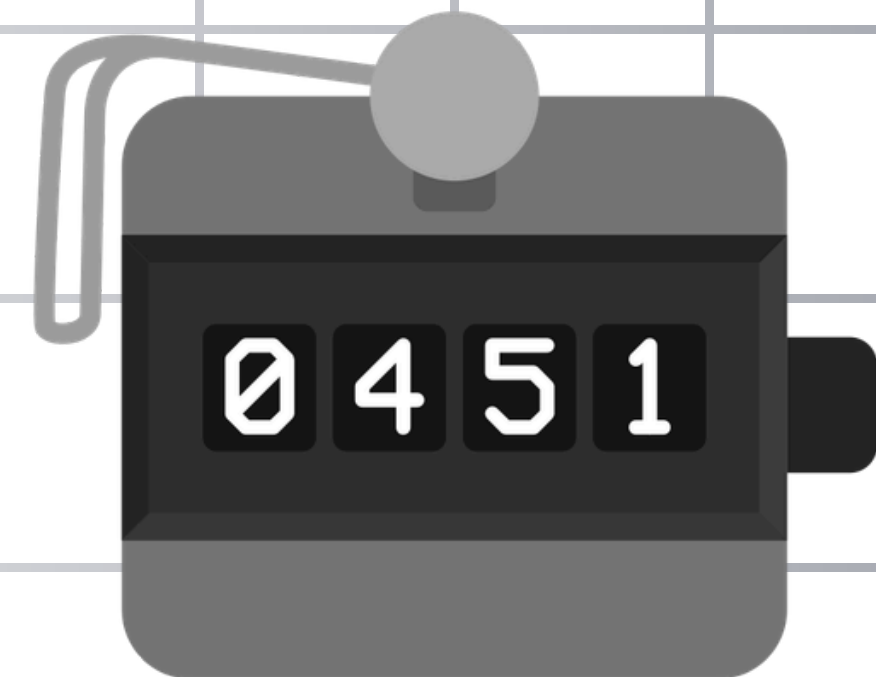
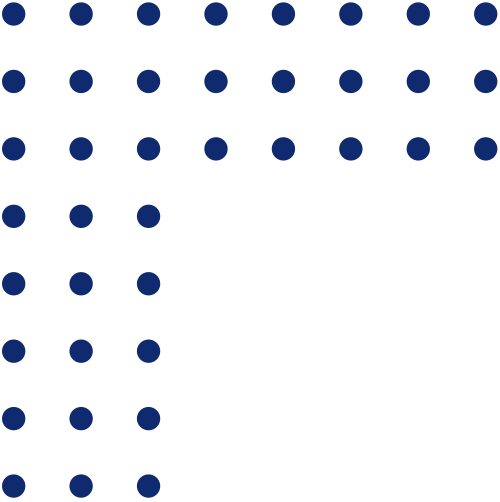




ANÁLISE EXPERIMENTAL DO
COUNTING SORT (RUST VS PYTHON)

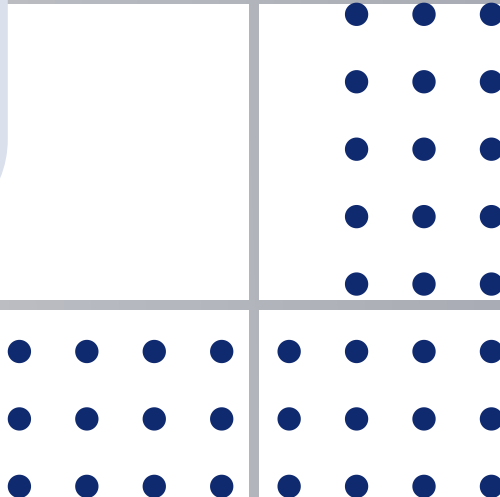
RUST VS PYTHON
TEORIA DA COMPUTAÇÃO
CESAR SCHOOL — EQUIPE 7

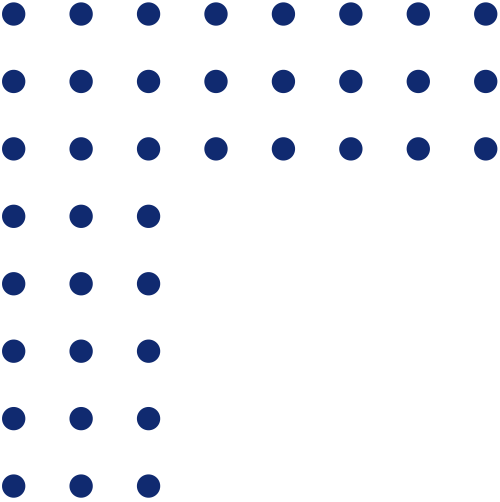




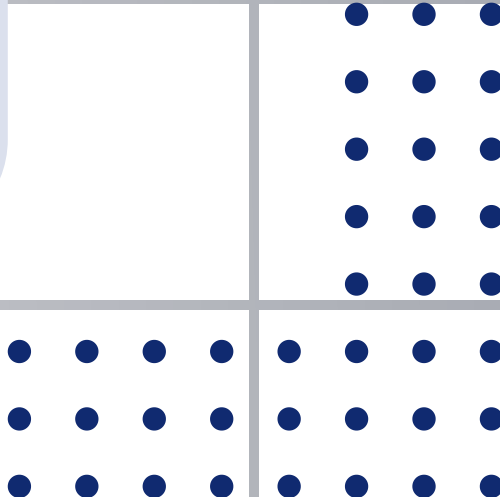
OBJETIVO



- Avaliar tempos de execução em Python e Rust no Algoritmo
 - Analisar a aderência entre complexidade teórica $O(n + k)$ e resultados práticos
 - Explorar os comportamentos e as características esperadas de cada linguagem nos experimentos.
 - Explorar o pior cenário do Counting Sort em ambas as linguagens
- 
- 



COMO FUNCIONA O COUNTING SORT?

- Algoritmo de ordenação não-comparativo linear.
 - Opera contando frequência de cada valor
 - Depende de um vetor auxiliar.
 - Melhor, pior e caso médio? $O(N + K)$
 - Sendo N = Elementos e K (Variação Numérica)
- 
- 
- 

FUNCIONAMENTO DO ALGORITMO

Input Data

0	4	2	2	0	0	1	1	0	1	0	2	4	2
---	---	---	---	---	---	---	---	---	---	---	---	---	---

Count Array

0	1	2	3	4
5	3	4	0	2

Sorted Data

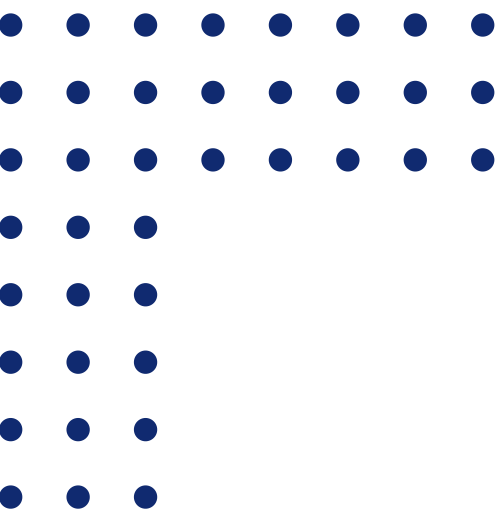
0	0	0	0	0	1	1	1	2	2	2	2	4	4
---	---	---	---	---	---	---	---	---	---	---	---	---	---

METODOLOGIA EXPERIMENTAL

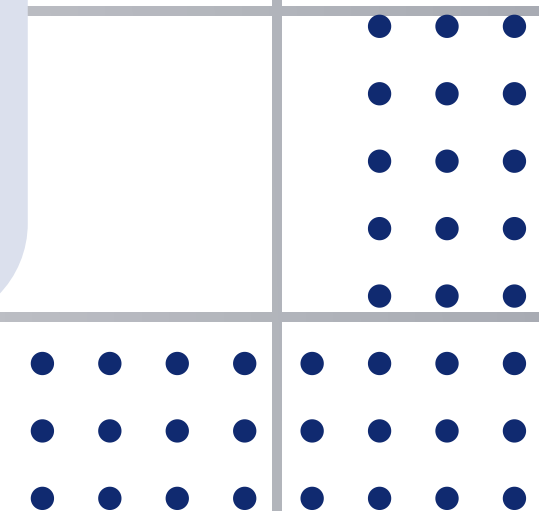
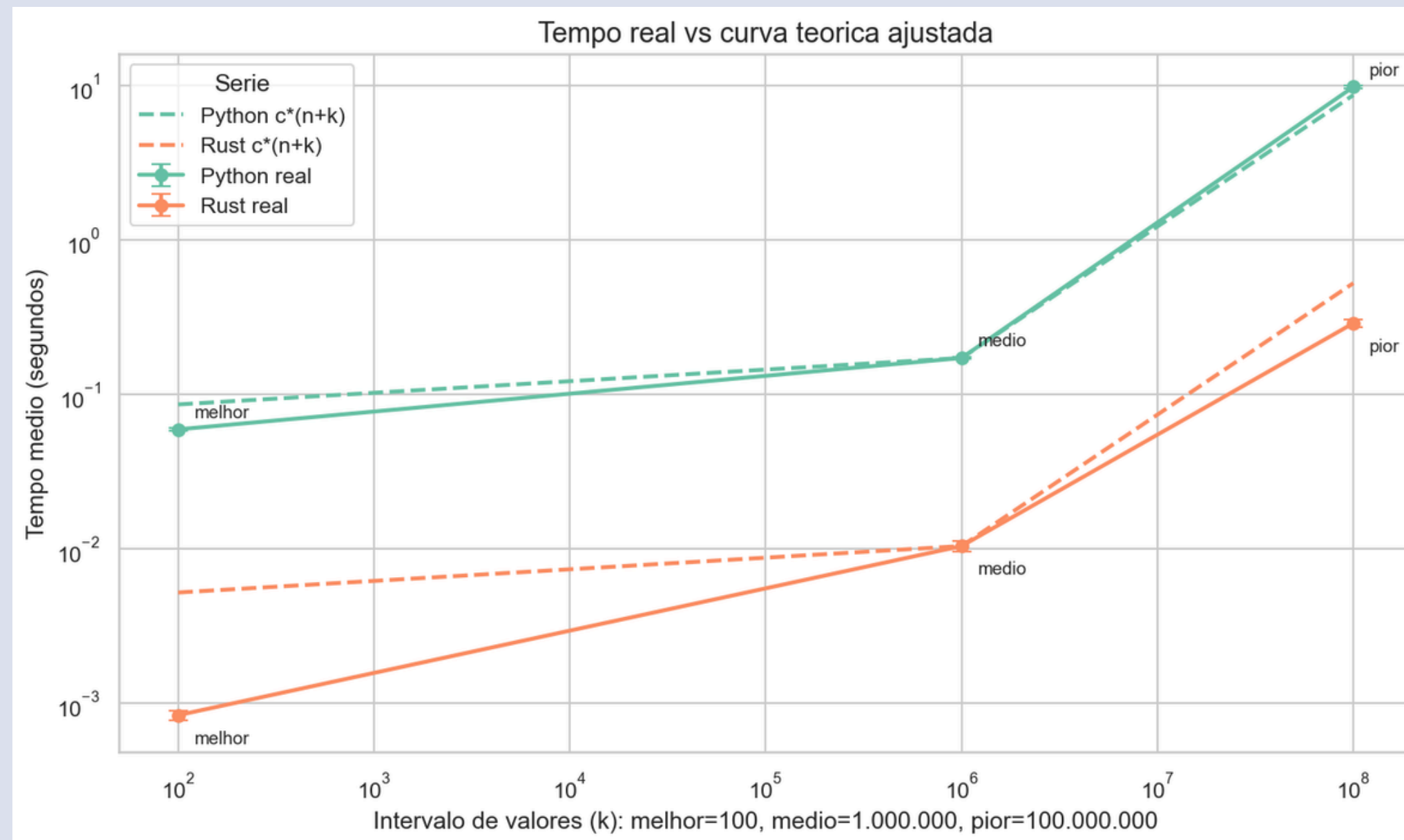
cenário	n	k
melhor	1.000.000	100
médio	1.000.000	1.000.000
pior	1.000.000	100.000.000

(30 Execuções em cada cenário)

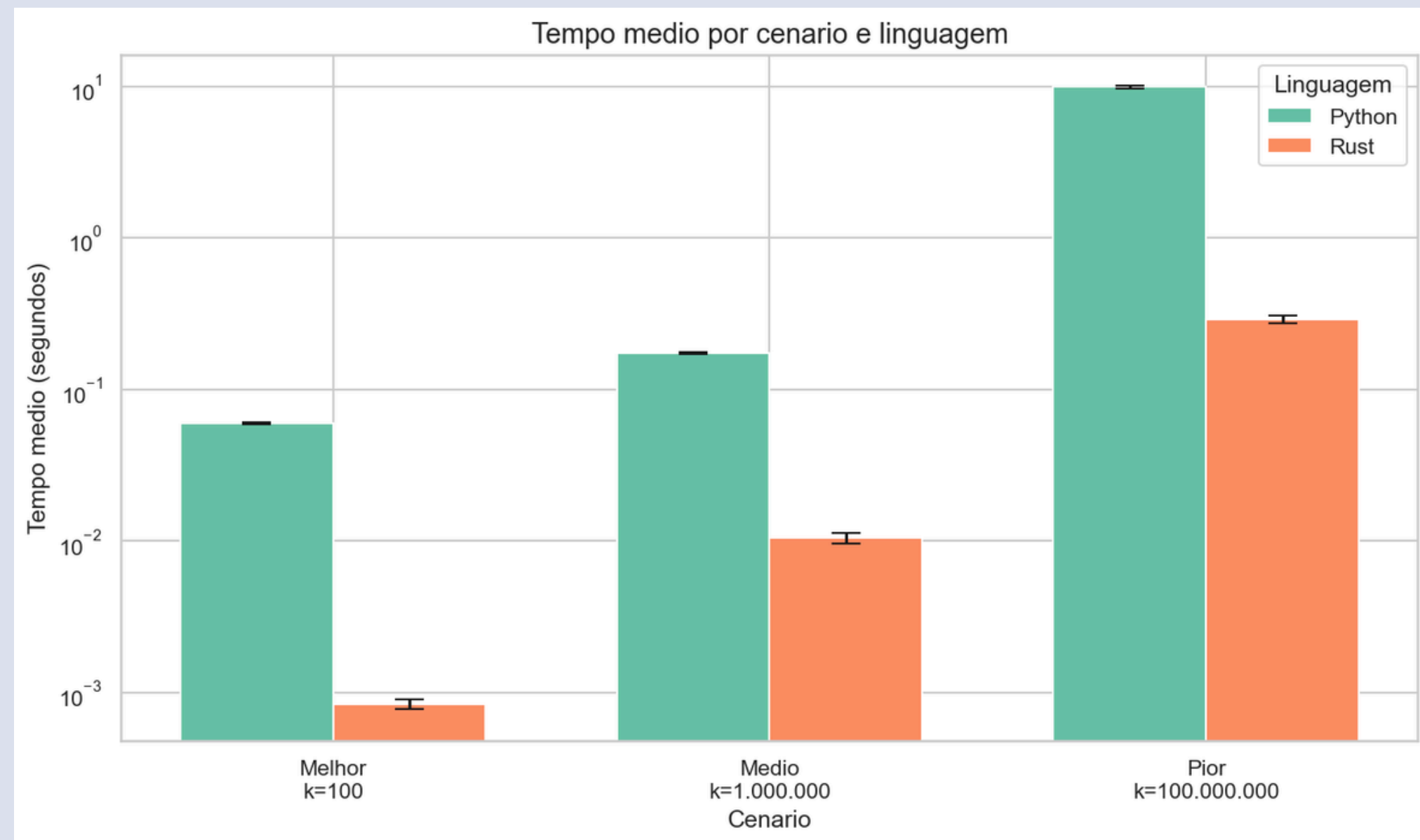
- Definimos nosso “pior” caso como o **K** (intervalo de valores) variando drasticamente, mantendo **N** fixo, já que a ordem inicial não altera o custo do algoritmo.
- Observamos que essa variação demanda mais recursos e tempo.
- Compartilharam a mesma seed para gerar entradas, validadas com checksums.



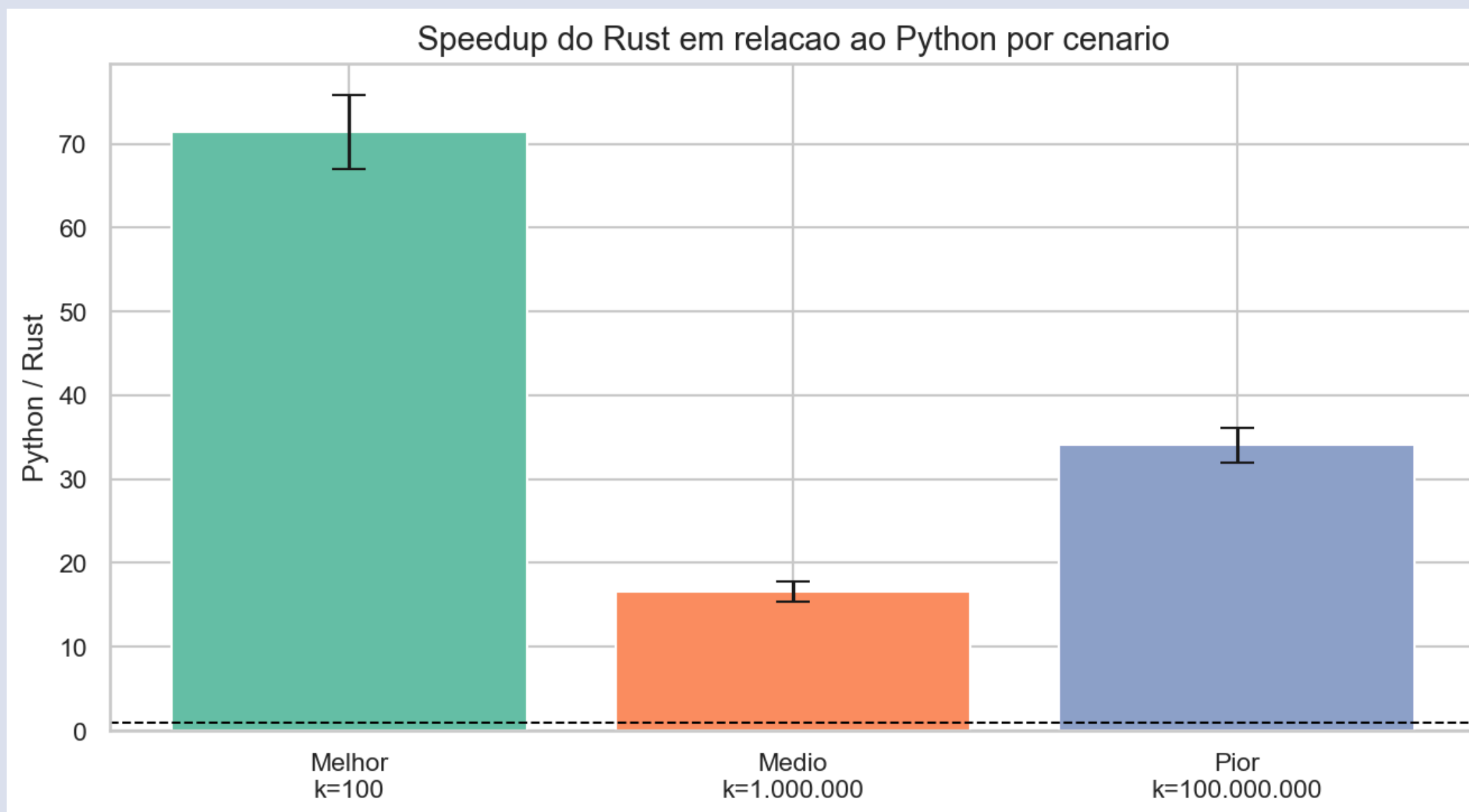
RESULTADOS: REAL VS TEÓRICO

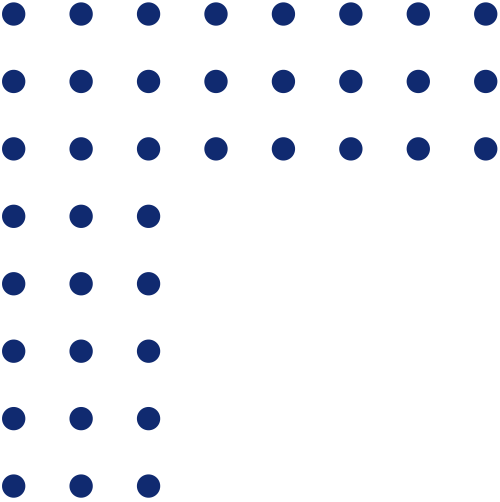


RESULTADOS: TEMPO MÉDIO DE EXECUÇÃO



RESULTADOS: VEREDICTO (SPEEDUP)



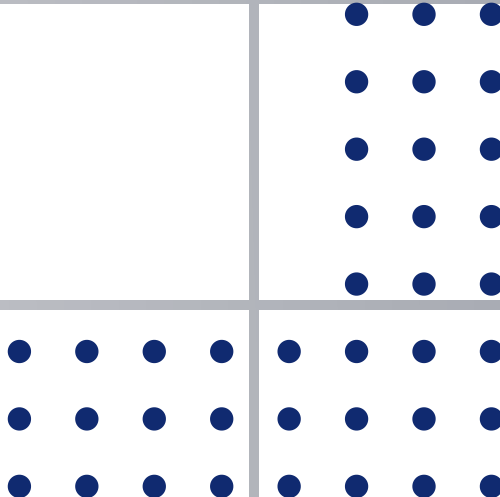


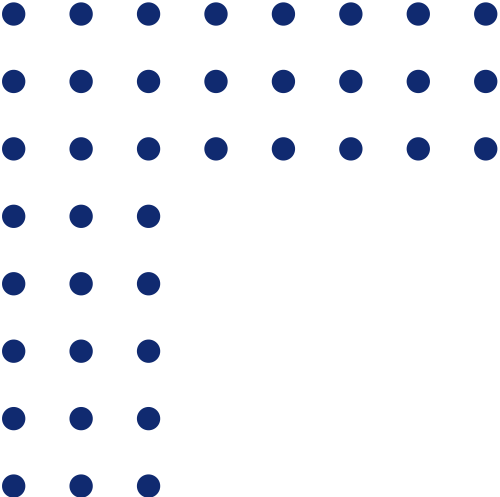
RESULTADOS: VEREDICTO (SPEEDUP)

K = 100 : Rust foi **71,1x** mais rápido que Python

K = 1.000.000 : Rust foi **16,5x** mais rápido que Python

K = 100.000.000: Rust foi **34,0x** mais rápido que Python

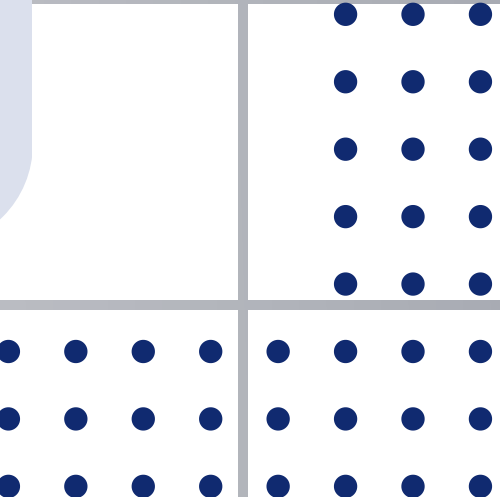


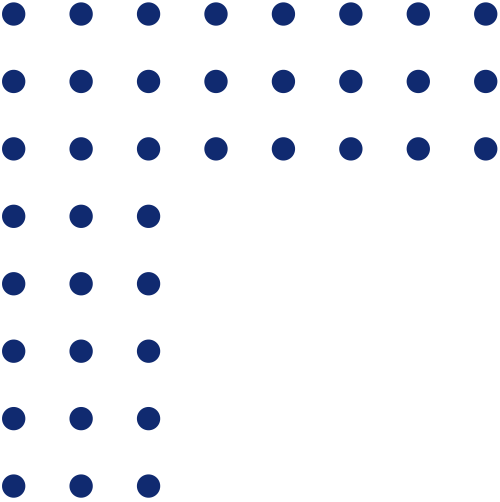


COMPARAÇÃO: PYTHON vs RUST

Python: Apresentou tempos maiores nos experimentos devido à maior sobrecarga de execução. Como é uma linguagem interpretada e de tipagem dinâmica, operações repetitivas e manipulação de listas impactam mais o desempenho do Counting Sort.

Rust: Apresentou tempos menores em todos os cenários. Por ser uma linguagem compilada e ter gerenciamento eficiente de memória, reduziu a sobrecarga na manipulação dos vetores auxiliares usados pelo Counting Sort.

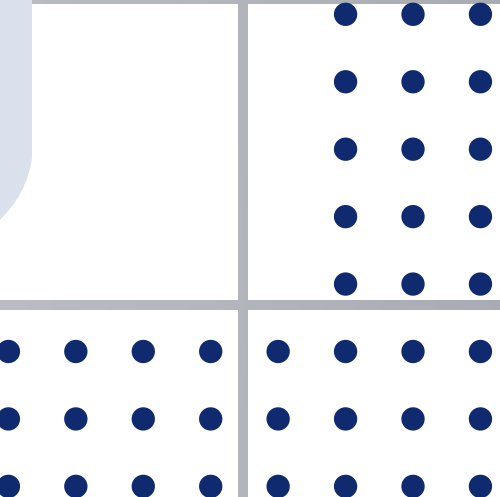


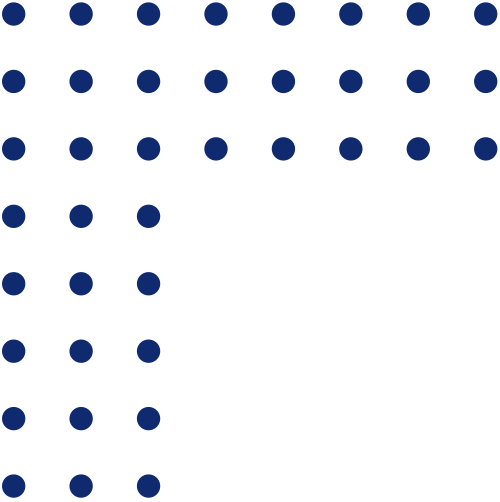


ADERÊNCIA TEÓRICA-PRÁTICA

Os resultados acompanharam a tendência esperada pela complexidade $\Theta(n + k)$. Como o número de elementos foi mantido constante, o aumento do tempo de execução ocorreu principalmente pelo crescimento de k .

A curva teórica não coincide perfeitamente com os tempos reais, pois fatores práticos como linguagem, alocação de memória, cache, sistema operacional e processos em segundo plano também influenciam as medições.

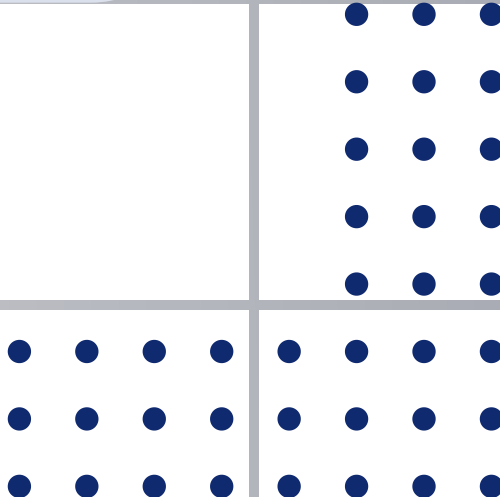


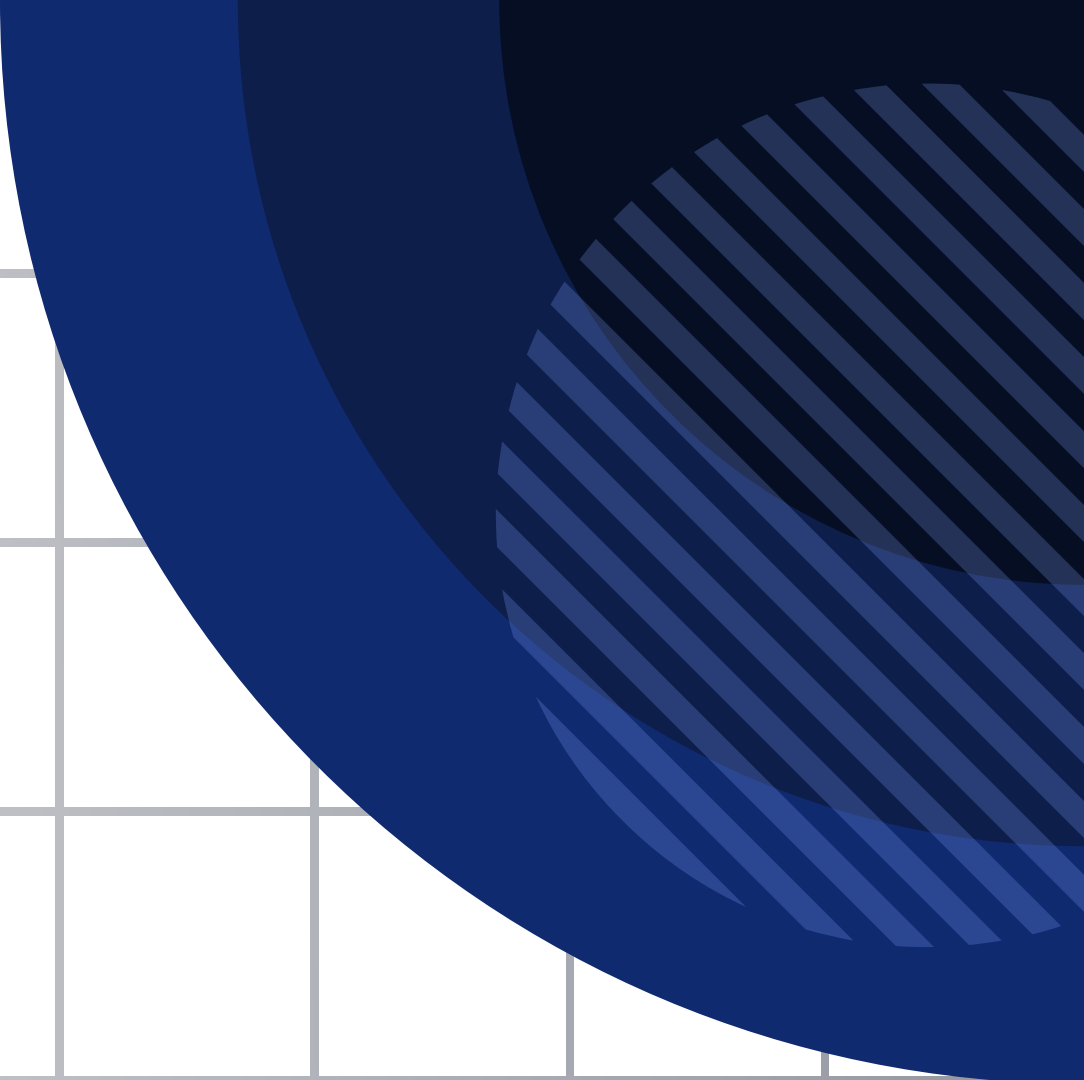
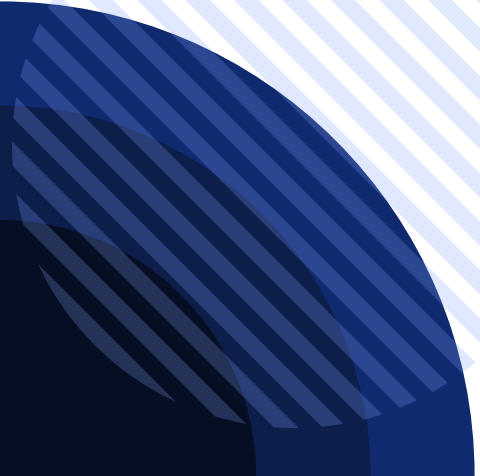
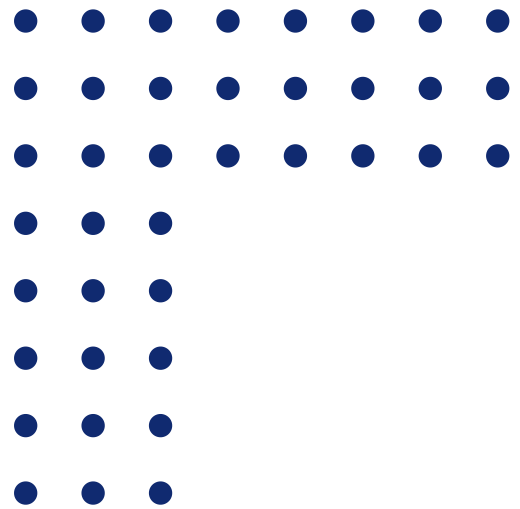
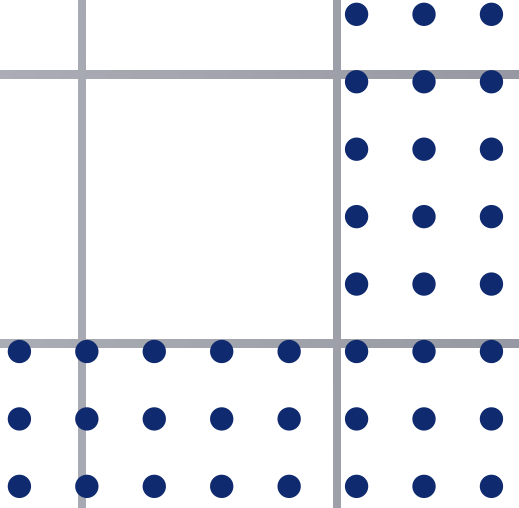


POR QUE PYTHON Vs RUST?

A comparação entre Python e Rust foi escolhida para representar o contraste entre produtividade e desempenho. Python é muito usado em IA e ciência de dados pela simplicidade e pelo ecossistema, enquanto Rust oferece execução eficiente, segurança de memória e maior controle sobre recursos.

Essa escolha também reflete o cenário atual: Python continua forte na prototipação e no uso de modelos, mas infraestruturas de alto desempenho tendem a depender de linguagens mais eficientes, como Rust, C ou C++.



A light gray grid pattern covers the entire background of the slide.A large, dark blue, curved shape with diagonal hatching is located in the top right corner.A blue and white striped, curved shape is located in the bottom left corner.A solid dark blue, curved shape is located in the bottom left corner, partially overlapping the striped shape.A grid of dark blue dots in the top left corner, arranged in 4 rows and 8 columns, with the bottom two rows being shorter (3 and 2 dots respectively).A grid of dark blue dots in the bottom right corner, arranged in 3 rows and 6 columns, with the top two rows being shorter (3 and 4 dots respectively).

OBRIGADO!

Arthur Xavier, Gustavo Oliveira, José Vitor e Talita Fraga